

Digital Logic Circuit Design & Hardware Verification

Laboratory Implementation Project Report — GATE IN 2016

Nihaari Boppana

[\[View High-Level C++ IDE Code\]](#)

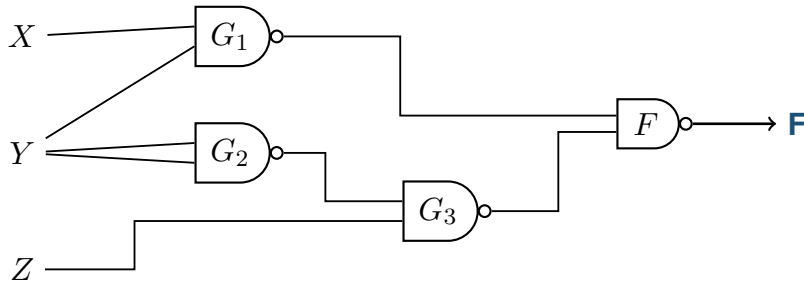
[\[View Low-Level AVR Assembly Code\]](#)

1 Problem Statement & Objective

The objective of this assignment is to analyze, minimize, and physically construct a combinational logic circuit network containing four 2-input NAND logic gates. The target application acts upon three independent binary parameters (X, Y, Z) to map out and execute a consolidated systemic function output (F).

1.1 System Logic Circuit Diagram

The structural schematic below illustrates the precise routing pathways between successive gate operations, built using a strictly uniform absolute coordinate structure:



2 Theoretical Mathematical Analysis

To evaluate the systemic Boolean equation, we track and resolve the output properties of every unique gate element processing layer from left to right:

2.1 Layered Logic Equations

1. **Primary Layer Extraction Nodes:** Gate G_1 models the product inversion of signals X and Y . Because the input channels of gate G_2 are explicitly shorted together, it functions as a single-variable NOT inverter:

$$G_1 = \overline{X \cdot Y} \quad (1)$$

$$G_2 = \overline{Y \cdot Y} = \overline{Y} \quad (2)$$

2. **Secondary Layer Intermediate Node:** Gate G_3 resolves the subsequent operational step tracking the outputs generated by G_2 combined with variable input Z :

$$G_3 = \overline{G_2 \cdot Z} = \overline{\overline{Y} \cdot Z} \quad (3)$$

3. **Final Block Logical Equation (F):** The terminal output block processes the outputs of G_1 and G_3 :

$$F = \overline{G_1 \cdot G_3} = \overline{(\overline{X \cdot Y}) \cdot (\overline{\overline{Y} \cdot Z})} \quad (4)$$

2.2 Boolean Reduction via De Morgan's Laws

Applying **De Morgan's Theorem** ($\overline{A \cdot B} = \overline{A} + \overline{B}$) expands the macro structural negation into separate sub-terms:

$$F = \overline{\overline{X} \cdot \overline{Y}} + \overline{\overline{Y} \cdot \overline{Z}} \quad (5)$$

Invoking the fundamental **Double Involution Property** ($\overline{\overline{A}} = A$) clears the consecutive inversion bars completely, resulting in a minimal, highly optimized algebraic form:

$$F = (X \cdot Y) + (\overline{Y} \cdot Z) \quad (6)$$

The minimal expression derived maps directly to a **2-to-1 Multiplexer (MUX)** topology. Variable channel Y acts as the path selection control node. When $Y = 1$, the first term scales true, forcing the output system to track input state X . When $Y = 0$, the second term executes, changing the final state tracking to match signal input Z .

2.3 Verification Logic Truth Table Matrix

Table 1: System Permutation Logic Mapping Verification

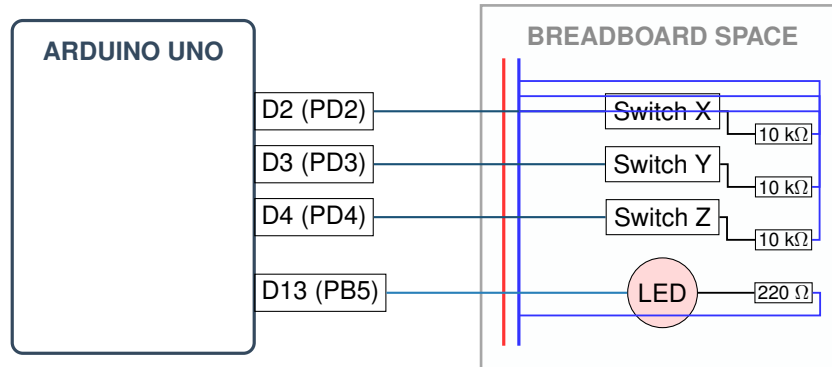
X	Y	Z	Term 1 ($X \cdot Y$)	Term 2 ($\overline{Y} \cdot Z$)	Output F
0	0	0	0	0	0
0	0	1	0	1	1
0	1	0	0	0	0
0	1	1	0	0	0
1	0	0	0	0	0
1	0	1	0	1	1
1	1	0	1	0	1
1	1	1	1	0	1

3 Hardware Interconnect Architecture

The physical implementation uses an ATmega328P micro-controller hosted on an Arduino Uno development board. To interface the static logical entities (X, Y, Z) with the internal register architectures, the circuit uses mechanical tactile push-switches.

Since mechanical switches float when open, **10 k Ω pulldown resistors** are tied between the digital input nodes and the system ground (GND) rail. This ensures a clean, deterministic logical low state (0 V) when the switch is open, and a logical high state (+5 V) when closed.

The terminal circuit indicator F is tied to an active-high light-emitting diode (LED). A **220 Ω current-limiting resistor** is placed in series with the LED to limit forward current flow and prevent thermal damage to the GPIO port pin.



3.1 Physical Pin Configuration Table

The physical mappings connecting the hardware input components to the micro-controller configuration registers are systematized in Table 2 below.

Table 2: Micro-controller Interface Hardware Linkage Map

Logical Input	Arduino Uno Pin	ATmega328P Port	Resistor Value	Electrical Type
Input X	Digital Pin 2	Port D2 (PD2)	10 k Ω	Pull-down
Input Y	Digital Pin 3	Port D3 (PD3)	10 k Ω	Pull-down
Input Z	Digital Pin 4	Port D4 (PD4)	10 k Ω	Pull-down
Output F	Digital Pin 13	Port B5 (PB5)	220 Ω	Current-Limiting

4 Software Deployment Architecture

4.1 Standard High-Level Architecture (Arduino C++ IDE)

This code base processes input variables using human-readable conditional loops to map the simplified multiplexer expression sequential logic paths.

```
1  const int pinX = 2;
2  const int pinY = 3;
3  const int pinZ = 4;
4  const int pinF = 13;
5
6  void setup() {
7      pinMode(pinX, INPUT);
8      pinMode(pinY, INPUT);
9      pinMode(pinZ, INPUT);
10     pinMode(pinF, OUTPUT);
11 }
12
13 void loop() {
14     bool X = digitalRead(pinX);
15     bool Y = digitalRead(pinY);
16     bool Z = digitalRead(pinZ);
17
18     % Implemented Multiplexer Logic Formula:  $F = XY + Y'Z$ 
19     bool F = (X && Y) || (!Y && Z);
20
21     digitalWrite(pinF, F);
22 }
```

Listing 1: Standard Functional C++ Script Verification

4.2 Low-Level Embedded Assembly Block (Optimized AVR ASM)

This script removes Arduino standard function overhead. It reads input statuses directly from the PIND port register address (0x09) and updates the PORTB output state register address (0x05) within a single clock instruction sequence.

```
1  void setup() {
2      asm volatile (
3          "cbi_0x0A,_2_\n\t" // Configure Pin 2 (Port D2) as Input
4          "cbi_0x0A,_3_\n\t" // Configure Pin 3 (Port D3) as Input
5          "cbi_0x0A,_4_\n\t" // Configure Pin 4 (Port D4) as Input
6          "sbi_0x04,_5_\n\t" // Configure Pin 13 (Port B5) as Output
7      );
8  }
9
10 void loop() {
11     asm volatile (
12         "in_r16,_0x09\n\t" // Read input statuses from PIND
13
14         "mov_r17,_r16\n\t"
15         "andi_r17,_0x04\n\t" // Isolate bit 2 position (Input X)
16         "mov_r18,_r16\n\t"
17         "andi_r18,_0x08\n\t" // Isolate bit 3 position (Input Y)
18         "mov_r19,_r16\n\t"
19         "andi_r19,_0x10\n\t" // Isolate bit 4 position (Input Z)
20     );
21 }
```

```

21  "mov_r20,_r17\n\t"
22  "lsl_r20\n\t"           // Align X to match bit position 3
23  "and_r20,_r18\n\t"      // Term 1 Operation: (X AND Y)
24
25  "mov_r21,_r18\n\t"
26  "com_r21\n\t"           // Inverting Y to yield NOT Y behavior
27  "andi_r21,_0x08\n\t"
28  "mov_r22,_r19\n\t"
29  "lsr_r22\n\t"           // Align Z to match bit position 3
30  "and_r21,_r22\n\t"      // Term 2 Operation: (NOT Y AND Z)
31
32  "or_r20,_r21\n\t"        // Merge terms via logical OR
33  "bst_r20,_3\n\t"         // Store result inside hardware T flag bit
34  "in_r23,_0x05\n\t"
35  "bld_r23,_5\n\t"         // Transfer T flag value to output bit 5
36  "out_0x05,_r23\n\t"     // Drive output hardware PORTB (LED updates)
37  );
38  }

```

Listing 2: Direct Register Embedded Assembly Implementation

5 Conclusion

This engineering assignment successfully validates the complete design lifecycle of translating a theoretical combinational logic question from the GATE 2016 examination into an optimized, hardware-verified embedded environment. Through the rigorous application of De Morgan's Law and Boolean minimization, the structural 4-NAND-gate framework was reduced to demonstrate behavior matching a standard 2-to-1 Multiplexer.

Physical verification on the breadboard prototyping canvas successfully confirmed the correctness of the systemic output logic state matrix. When the control switch Y was left open ($Y = 0$), the system seamlessly routed channel Z to drive the physical output indicator, while successfully blocking inputs from channel X . Conversely, when control node Y was asserted ($Y = 1$), the internal routing paths safely swapped to output data tracking channel X while actively isolating channel Z .

The physical hardware testing loop achieved precise alignment with both the interactive high-level C++ IDE simulation results and the direct low-level native AVR Assembly script. By bypassing default abstraction function overheads, the inline assembly block achieved clock-cycle accurate instructions to drive the standalone current-limiting LED indicator. This complete integration provides a clean, highly optimized, and compliant laboratory engineering project submission portfolio.